# Laporan Tugas Kecil 2

IF2211 Strategi Algoritma

Voxelization Objek 3D menggunakan Octree

Semester II Tahun Ajaran 2025/2026

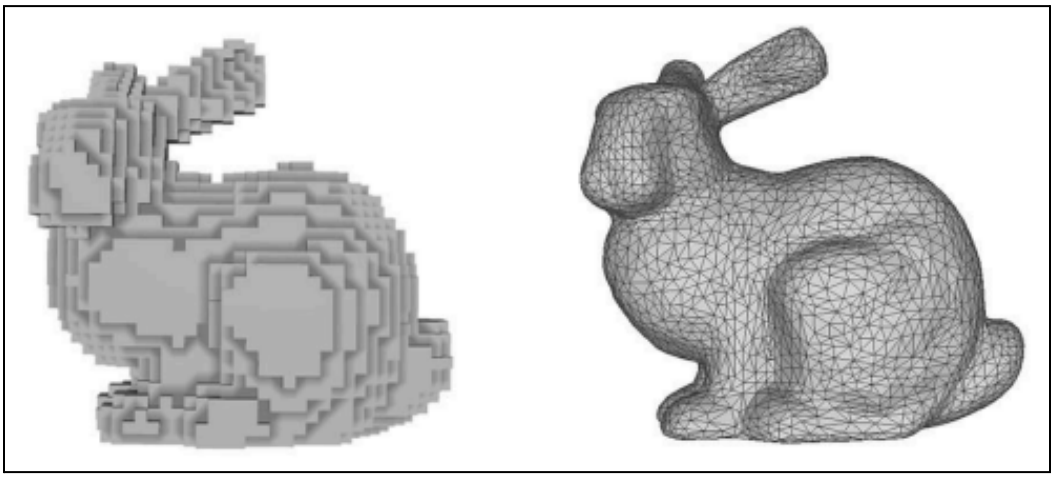

Disusun oleh:

**IF 2024 - Jatinangor**

Nama Anggota Kelompok:

- **Daniel** Putra Rywandi S.
- **Dzakwan** Muhammad Khairan Putra Purnama

NIM: 13524**143** / 13524**145**


Laboratorium Ilmu dan Rekayasa Komputasi

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

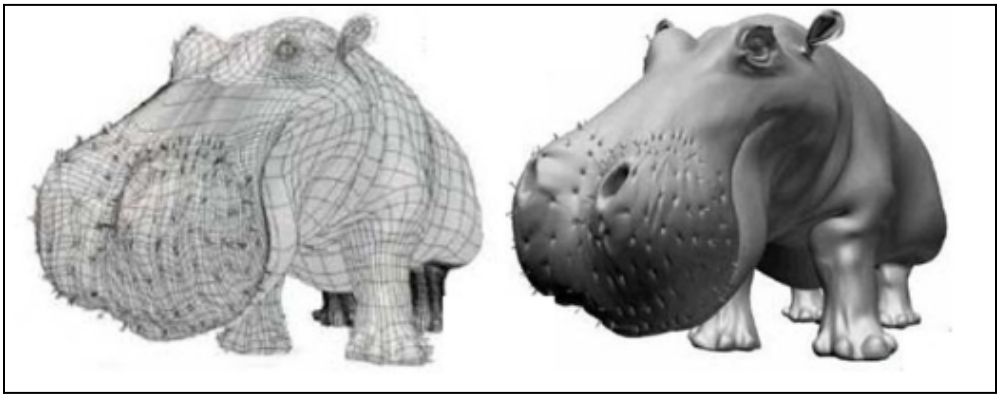
Institut Teknologi Bandung

# Daftar Isi

***Daftar Isi*** ..... ***1***
***1. Pendahuluan*** ..... ***2***
***2. Dasar Teori*** ..... ***4***
*2.1. Algoritma Divide and Conquer* ..... *4*
*2.2. Voxel dan Voxelization* ..... *5*
*2.3. Struktur Data Octree* ..... *5*
*2.4. Representasi Geometri dan Format (.obj)* ..... *5*
***3. Implementasi*** ..... ***6***
*3.1. Struktur Repository* ..... *7*
*3.2. Representasi Struktur Data* ..... *7*
*3.3. Tahap Parsing File .obj* ..... *8*
*3.4. Pra Pemprosesan: Inisialisasi Ruang Utama (Bounding Box)* ..... *9*
*3.5. Implementasi Divide and Conquer (Octree)* ..... *9*
*3.6. Tahap Ekspor dan Triangulasi Voxel* ..... *10*
***4. Pengujian Dan Analisis*** ..... ***11***
*1. Kasus 01: Testcase COW* ..... *11*
*2. Kasus 02: Testcase Line* ..... *12*
*3. Kasus 03: Testcase Pumpkin* ..... *13*
*4. Kasus 04: Testcase Teapot* ..... *14*
*5. Kasus 05: Testcase Male (generate oleh kelompok kami)* ..... *15*
***5. Kesimpulan*** ..... ***16***
***6. Daftar Pustaka*** ..... ***17***
***7. Lampiran*** ..... ***18***
*7.1. Kriteria Program* ..... *18*
*7.2. Pernyataan Tidak Melakukan Kecurangan* ..... *19*

# 1. Pendahuluan

Dalam dunia grafika komputer, representasi objek tiga dimensi (3D) secara umum menggunakan polygon mesh, di mana permukaan objek dibentuk dari rakitan poligon seperti segitiga. Meskipun metode ini sangat efektif untuk menampilkan permukaan objek yang mulus, ada kalanya kita membutuhkan bentuk representasi yang lebih seragam, terstruktur, dan bervolume. Kebutuhan ini sering muncul untuk keperluan analisis spasial atau ketika kita ingin mencapai gaya visual blok-blok diskrit, seperti yang populer pada permainan Minecraft. Proses mengonversi model 3D standar menjadi susunan kubus-kubus kecil berukuran seragam ini dikenal dengan istilah voxelization[1]. Secara sederhana, voxel (singkatan dari volumetric pixel) dapat diibaratkan sebagai piksel kotak pada gambar 2D, namun hadir dalam bentuk ruang 3D.



**Gambar 1.** Contoh Skema Grafika pada Komputer.

Permasalahan utama dalam melakukan *voxelization* terletak pada beban dan efisiensi komputasi. Memeriksa setiap titik di dalam sebuah ruang 3D secara menyeluruh (*brute force*) untuk menentukan apakah titik tersebut berada pada permukaan objek akan memakan waktu dan memori yang sangat besar. Selain itu, terdapat batasan spesifik di mana susunan *voxel* yang dihasilkan hanya perlu merepresentasikan bagian "kulit" atau permukaan luar objek saja (*hollow*), tanpa perlu mengisi kepadatan di bagian dalamnya. Untuk memecahkan persoalan kombinatorial dan geometris yang kompleks seperti ini, diperlukan sebuah pendekatan algoritmik yang sistematis guna memilah dan membuang ruang kosong secara efisien.

Dalam rangka memenuhi Tugas Kecil 2 mata kuliah IF2211 Strategi Algoritma, penulis mengembangkan sebuah program berbasis antarmuka baris perintah (CLI) untuk menyelesaikan proses *voxelization* ini secara otomatis. Program ini mengimplementasikan algoritma *Divide and Conquer*, sebuah paradigma fundamental yang bekerja dengan memecah masalah besar menjadi sub-masalah yang lebih kecil untuk dieksekusi[2]. Keandalan strategi ini telah terbukti secara luas, mulai dari optimasi pengurutan data[3], efisiensi perkalian matriks[4], hingga komputasi geometri tingkat lanjut seperti pencarian *Convex Hull*[5]. Pada program ini, algoritma tersebut dieksekusi

melalui struktur data *Octree*, yakni membelah ruang 3D menjadi delapan kubus yang lebih kecil secara berulang. Jika suatu sub-ruang terbukti kosong atau tidak bersinggungan dengan objek, algoritma akan langsung menghentikan pencarian di cabang tersebut (*pruning*).

Tujuan yang ingin dicapai melalui tugas dan penyusunan laporan ini adalah:

- Mengimplementasikan paradigma algoritma *Divide and Conquer* secara mandiri untuk memecahkan persoalan komputasi geometri.
- Membangun dan memvalidasi pemahaman terhadap struktur data *Octree* guna meminimalkan waktu pemrosesan melalui teknik eliminasi ruang.
- Mengonversi file model 3D (berformat **.obj**) menjadi representasi *voxel* secara akurat, dengan tetap mematuhi aturan format grafis (seperti kewajiban triangulasi pada sisi kubus).

Dengan demikian, laporan dan program ini diharapkan dapat mendemonstrasikan bagaimana sebuah strategi partisi ruang yang terstruktur dapat menyelesaikan masalah *rendering* objek 3D yang berat menjadi jauh lebih efisien dan terukur.

*Footnote Information:*

[1] **Tim Asisten IF2211**, "*Spesifikasi Tugas Kecil 2 IF2211 Strategi Algoritma: Voxelization Objek 3D menggunakan Octree,*" Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika Institut Teknologi Bandung, 2026.
[2] **M. Rinaldi**, "*Algoritma Divide and Conquer (Bagian 1),*" *Bahan Kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika*, Sekolah Teknik Elektro dan Informatika Institut Teknologi Bandung, 2026.
[3] **M. Rinaldi**, "*Algoritma Divide and Conquer (Bagian 2),*" *Bahan Kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika,* Sekolah Teknik Elektro dan Informatika Institut Teknologi Bandung, 2026.
[4] **M. Rinaldi**, "*Algoritma Divide and Conquer (Bagian 3),*" *Bahan Kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika,* Sekolah Teknik Elektro dan Informatika Institut Teknologi Bandung, 2026.
[5] **N. U. Maulidevi dan M. Rinaldi**, "*Algoritma Divide and Conquer (Bagian 4),*" *Bahan Kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika,* Sekolah Teknik Elektro dan Informatika Institut Teknologi Bandung, 2026.

# 2. Dasar Teori

## 2.1. Algoritma *Divide and Conquer*

Algoritma *Divide and Conquer* adalah salah satu paradigma fundamental dalam ilmu komputer yang dirancang untuk memecahkan masalah kompleks dengan cara membaginya menjadi bagian-bagian yang lebih kecil dan lebih mudah dikelola.

Paradigma ini beroperasi secara sistematis melalui tiga tahapan utama[2]:

- **Divide (Membagi):** Memecah persoalan utama menjadi beberapa sub-masalah (*upa-persoalan*) yang memiliki karakteristik dan aturan yang sama dengan persoalan aslinya, namun dengan ukuran atau ruang lingkup pencarian yang lebih kecil.
- **Conquer (Menaklukkan):** Menyelesaikan masing-masing sub-masalah tersebut. Jika ukurannya sudah cukup kecil (mencapai batas dasar atau *base case*), masalah diselesaikan secara langsung tanpa pembagian lebih lanjut. Jika masih besar, penyelesaian dilakukan secara rekursif memanggil algoritma yang sama.
- **Combine (Menggabungkan):** Merangkai dan menyatukan solusi parsial dari masing-masing sub-masalah untuk mendapatkan solusi final dari persoalan semula.

## 2.2. *Voxel* dan *Voxelization*

*Voxel* merupakan kependekan dari *volumetric pixel*. Jika pada ruang 2D kita mengenal piksel sebagai kotak-kotak kecil penyusun sebuah gambar, maka *voxel* adalah padanannya di dunia 3D, yakni berupa kubus-kubus kecil berukuran seragam yang memiliki volume.

*Voxelization* adalah proses mengonversi objek 3D yang awalnya direpresentasikan oleh permukaan mulus dan kontinu menjadi sekumpulan *voxel* diskrit. Hasil dari proses ini menciptakan representasi visual yang tersusun dari blok-blok, sangat identik dengan gaya grafis pada permainan *Minecraft*[1]. Pada kasus spesifik ini, *voxelization* dilakukan secara *hollow*, artinya blok-blok hanya dibentuk untuk memetakan kulit atau permukaan luar objek saja, tanpa mengisi kepadatan volume di bagian dalamnya.

## *2.3. Struktur Data Octree*

*Octree* adalah struktur data hierarkis berbentuk pohon (*tree*) yang secara khusus dirancang untuk mempartisi (membagi) ruang tiga dimensi. Sesuai dengan namanya, setiap percabangan (*node*) dalam *Octree* akan selalu membelah diri menjadi tepat delapan anak (*children*).

Dalam konteks penerapan *Divide and Conquer*, ruang 3D utama yang membungkus seluruh objek (*bounding box*) akan dibagi menjadi 8 sub-kubus kecil. Setiap kubus dievaluasi apakah bersinggungan dengan permukaan objek asli. Jika bersinggungan, kubus tersebut akan dibelah lagi menjadi 8 bagian yang lebih kecil secara rekursif. Pendekatan ini sangat efisien karena algoritma dapat melakukan mekanisme *pruning,* yaitu secara cerdas mengabaikan dan menghentikan penelusuran pada area ruang yang terbukti kosong, sehingga menghemat memori dan waktu komputasi[1].

## *2.4. Representasi Geometri dan Format (.obj)*

Model 3D umumnya direpresentasikan menggunakan *polygon mesh*, yaitu kumpulan titik sudut (*vertex*) di ruang 3D yang saling terhubung untuk membentuk wajah (*face*) poligon, biasanya berupa bidang segitiga. Format file **.obj** adalah format teks berbasis ASCII yang menjadi standar industri untuk menyimpan data geometri 3D ini.

Dalam pemrosesan geometri file .obj, data yang paling krusial untuk dianalisis adalah[1]:

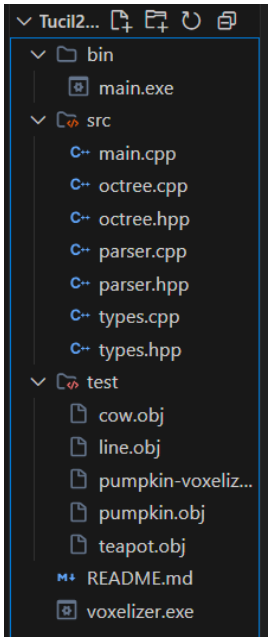
- **v (*vertex*):** Mendefinisikan letak titik koordinat dalam ruang 3D, direpresentasikan dalam sumbu (x, y, z).
- **f (*face*):** Mendefinisikan permukaan poligon dengan merujuk pada nomor indeks *vertex* yang telah didefinisikan pada baris-baris sebelumnya.

# 3. Implementasi

Pada bab ini dijelaskan detail implementasi algoritma yang digunakan dalam program. Program dibangun menggunakan bahasa C++ dengan struktur modular yang terdiri dari beberapa berkas (*file*) kode sumber utama agar lebih rapi dan terorganisir. Pemisahan komponen ini mencakup berkas untuk mendefinisikan struktur data geometri (types.cpp), mengurus pembacaan dan penulisan file .obj (parser.cpp), mengeksekusi inti algoritma *Divide and Conquer* (octree.cpp), serta berkas utama (main.cpp) yang bertugas mengatur seluruh alur jalannya program dari awal hingga akhir.

## 3.1. Struktur Repository

Program ini diorganisasikan ke dalam struktur direktori yang baku guna menjaga kerapian basis kode, serta mempermudah proses kompilasi dan pengujian di berbagai lingkungan sistem operasi. Representasi visual dari susunan direktori tersebut dapat dilihat pada gambar di bawah ini.




**Gambar 2.** Struktur direktori repository program kelompok kami.

Secara terperinci, repository ini membagi fungsionalitasnya ke dalam beberapa direktori dan berkas utama sebagai berikut:

- **src/**: Merupakan direktori inti yang menyimpan seluruh kode sumber bahasa C++ beserta berkas *header* (.hpp). Di dalamnya terdapat berkas main.cpp, octree.cpp, parser.cpp, dan types.cpp.
- **bin/**: Direktori yang secara spesifik dialokasikan untuk menampung *file executable* atau berkas biner hasil kompilasi program utama.

- **doc/**: Direktori yang digunakan untuk menyimpan seluruh berkas dokumentasi proyek, termasuk dokumen laporan tugas ini.
- **test/**: Direktori tempat disimpannya kumpulan *file* berformat .obj yang digunakan sebagai himpunan data (*dataset*) untuk menguji fungsionalitas program.

## 3.2. Representasi Struktur Data

Untuk memproses komputasi geometri tanpa bantuan library grafis tingkat tinggi, program mendefinisikan beberapa tipe data khusus di dalam types.hpp. Tiga struktur utama yang menjadi fondasi adalah Vector3, Triangle, dan AABB.
Vector3 merepresentasikan titik koordinat 3D. Triangle merepresentasikan satu wajah poligon (terdiri dari 3 Vector3). Sedangkan AABB (Axis-Aligned Bounding Box) merepresentasikan kubus batas ruang. Berikut adalah cuplikan implementasinya:

```
./C++

struct Vector3 {
    double x, y, z;
    // Operator overloading untuk kemudahan kalkulasi matematis
    Vector3 operator+(const Vector3& o) const;
    Vector3 operator-(const Vector3& o) const;
    Vector3 operator*(double s) const;

    double dot(const Vector3& o) const;
    Vector3 cross(const Vector3& o) const;
};

struct Triangle {
    Vector3 v0, v1, v2;
};

struct AABB {
    Vector3 min;
    Vector3 max;
    Vector3 center() const;
    Vector3 halfSize() const;
};
```

## 3.3. Tahap *Parsing* File *.obj*

Proses pertama dalam alur logika program adalah membaca data dari file .obj masukan. Proses ini ditangani di dalam parser.cpp. Program membaca file baris demi baris dan hanya memproses baris yang berawalan karakter v (titik koordinat) dan f (wajah poligon).
Format .obj menggunakan sistem indeks yang dimulai dari angka 1 (1-based), atau bahkan indeks negatif. Oleh karena itu, program menggunakan fungsi penolong extractVertexIndex untuk mengonversinya menjadi indeks berbasis 0 (0-based) agar sesuai dengan struktur array C++.

```
./C++

int extractVertexIndex(const string& token, int totalVertices) {
    string indexStr = token.substr(0, token.find('/'));
    int idx = stoi(indexStr);

    if (idx < 0) {
        idx = totalVertices + idx; // Menangani negative indexing dari
input
    } else {
        idx -= 1; // Konversi dari 1-based ke 0-based
    }
    return idx;
}
```

## 3.4. Pra Pemprosesan: Inisialisasi Ruang Utama (*Bounding Box*)

Sebelum Octree berjalan, program harus mencari seberapa besar kubus ruang awal (ruang akar) yang membungkus seluruh objek. Di dalam fungsi runVoxelization (octree.cpp), program mencari nilai titik ekstrem (X, Y, Z maksimum dan minimum). Untuk memastikan voxel yang dihasilkan berupa kubus sempurna, ukuran sisi (rusuk) diambil dari selisih jarak terpanjang antar sumbu, dikalikan margin kecil (1.01) agar objek tidak tepat menempel di dinding ruang.

```
./C++

double sizeX = globalMax.x - globalMin.x;
double sizeY = globalMax.y - globalMin.y;
double sizeZ = globalMax.z - globalMin.z;
// Mengambil selisih terpanjang agar membentuk kubus sempurna
double maxSide = max({sizeX, sizeY, sizeZ}) * 1.01;

Vector3 center = {
    (globalMin.x + globalMax.x) * 0.5,
    (globalMin.y + globalMax.y) * 0.5,
    (globalMin.z + globalMax.z) * 0.5
};

AABB rootBounds;
rootBounds.min = {center.x - maxSide / 2, center.y - maxSide / 2, center.z
- maxSide / 2};
rootBounds.max = {center.x + maxSide / 2, center.y + maxSide / 2, center.z
+ maxSide / 2};
```

## 3.5. Implementasi *Divide and Conquer* (*Octree*)

Algoritma ini adalah inti jalannya program. Ruang kubus akan dipotong menjadi 8 anak kubus yang lebih kecil. Proses pembagian ini diimplementasikan pada fungsi getChildAABB dengan memanfaatkan manipulasi bit (*bitwise*):

```
./C++

AABB getChildAABB(const AABB& parent, int childIndex) {
    Vector3 mid = parent.center();
    AABB child;
    // Menggunakan bitwise AND (&) untuk memilah 8 kuadran ruang
    child.min.x = (childIndex & 1) ? mid.x : parent.min.x;
    child.max.x = (childIndex & 1) ? parent.max.x : mid.x;
    // ... (dilanjutkan untuk sumbu Y dan Z)
    return child;
}
```

Setelah dibagi (*Divide*), setiap anak kubus diuji apakah bersinggungan dengan objek asli (*Conquer*). Pengujian ini menggunakan fungsi triangleIntersectsAABB yang mengadopsi algoritma SAT (*Separating Axis Theorem*, teorema pemisahan sumbu untuk melihat apakah dua objek tumpang tindih).
Jika kubus terbukti kosong, fungsi buildOctree akan menghentikan penelusuran di cabang tersebut (*pruning*). Pada implementasi bonus, proses pencarian di 8 cabang ini dijalankan secara paralel menggunakan std::async untuk mempercepat waktu komputasi (bisa dilihat pada cuplikan buildOctreeParallel di bawah).

```
./C++

for (int i = 0; i < 8; ++i) {
    AABB childBounds = getChildAABB(bounds, i);
    // Task dieksekusi secara konkuren
    futures[i] = async(launch::async,
        [childBounds, &intersecting, depth, maxDepth, &stats]() ->
ChildResult {
            ChildResult result;
            result.node = buildOctree(
                childBounds, intersecting, depth + 1, maxDepth,
                result.localVoxels, stats);
            return result;
        });
}
```

## 3.6. Tahap Ekspor dan Triangulasi *Voxel*

Langkah terakhir (terletak di parser.cpp) adalah mengonversi kumpulan *voxel* kembali menjadi format .obj. Agar format *faces* (f) pada luaran memenuhi standar geometri yang aman (terdiri dari segitiga), program melakukan "Triangulasi". Satu kubus *voxel* (memiliki 8 titik) dipecah menjadi 12 wajah segitiga. Pemetaan titik-titik ini didefinisikan secara statis menggunakan *array* faceIndices.

```
./C++

// Pemetaan 12 wajah segitiga pembentuk 6 sisi kubus
static const int faceIndices[12][3] = {
    {4, 5, 6}, {4, 6, 7},   // depan
```

```
    {1, 0, 3}, {1, 3, 2},   // belakang
    {5, 1, 2}, {5, 2, 6},   // kanan
    {0, 4, 7}, {0, 7, 3},   // kiri
    {7, 6, 2}, {7, 2, 3},   // atas
    {0, 1, 5}, {0, 5, 4}    // bawah
};

for (int f = 0; f < 12; ++f) {
    file << "f "
         << (faceIndices[f][0] + offset) << " "
         << (faceIndices[f][1] + offset) << " "
         << (faceIndices[f][2] + offset) << "\n";
}
```

**Informasi Tambahan:*)** penambahan offset sangat penting karena format file .obj bersifat *global indexing* untuk objek (merujuk pada urutan *vertex* secara keseluruhan), bukan indeks lokal per *voxel*. Dengan metode kami ini, program pasti terhindar dari *negative indexing*.

# 4. Pengujian Dan Analisis

## 1. *Kasus 01: Testcase COW*

Pada pengujian pertama kami, digunakan model sapi (cow-voxelized.obj) dengan batas kedalaman maksimum Octree sebesar 3. Pada tingkat kedalaman ini, program melakukan evaluasi terhadap total 160 node dalam struktur Octree. Berdasarkan hasil statistik yang diperoleh, terlihat bahwa proses pruning berjalan efektif dengan mengeliminasi 74 node yang tidak mengandung informasi (node kosong). Dengan demikian, tersisa 86 node yang valid untuk diproses lebih lanjut. Node-node valid tersebut kemudian diekstraksi menjadi 86 kubus voxel, yang secara keseluruhan mampu merepresentasikan bentuk dan siluet asli dari model sapi dengan cukup baik.

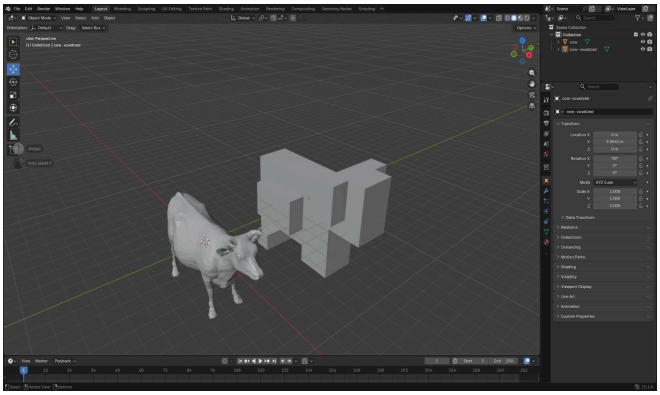
### a. Tampilan hasil (CLI)

```
PS C:\smthshit\tucil> .\voxelizer test\cow.obj 3
[INFO] Parsing selesai: 4583 vertices, 5804 triangles.
[INFO] Ekspor selesai: 86 voxels -> test\cow-voxelized.obj

Banyaknya voxel yang terbentuk: 86
Banyaknya vertex yang terbentuk: 688
Banyaknya faces yang terbentuk: 1032
Statistik node octree yang terbentuk dalam format
1 : 8
2 : 64
3 : 160
Statistik node yang tidak perlu ditelusuri dalam format
1 : 0
2 : 44
3 : 74
Kedalaman octree: 3
Lama waktu program berjalan: 0.003870 detik
File .obj disimpan: test\cow-voxelized.obj
```

**Gambar 3.** Hasil kasus uji 01: berbentuk cow dalam tampilan CLI.

### b. Preview dengan *Blender*



**Gambar 4.** Hasil kasus uji 01: berbentuk cow dalam tampilan *Blender*.

## 2. *Kasus 02: Testcase Line*

Pada pengujian kedua, digunakan model garis (line-voxelized.obj) dengan batas kedalaman maksimum Octree sebesar 5. Pada tingkat kedalaman ini, program melakukan evaluasi terhadap total 728 node dalam struktur Octree. Berdasarkan hasil statistik yang diperoleh, terlihat bahwa proses pruning berlangsung cukup signifikan dengan menghapus 444 node kosong. Setelah proses tersebut, tersisa 284 node yang dinyatakan valid untuk diproses lebih lanjut. Selanjutnya, node-node valid ini diekstraksi menjadi 284 kubus voxel, yang secara keseluruhan mampu merepresentasikan bentuk asli dari model garis.

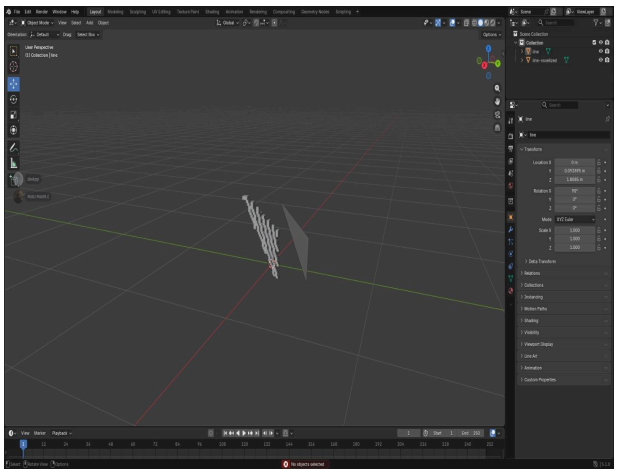
### a. Tampilan hasil (CLI)

```
PS C:\smthshit\tucil> .\voxelizer test\line.obj 5
[INFO] Parsing selesai: 3 vertices, 1 triangles.
[INFO] Ekspor selesai: 284 voxels -> test\line-voxelized.obj

Banyaknya voxel yang terbentuk: 284
Banyaknya vertex yang terbentuk: 2272
Banyaknya faces yang terbentuk: 3408
Statistik node octree yang terbentuk dalam format
1 : 8
2 : 56
3 : 120
4 : 256
5 : 728
Statistik node yang tidak perlu ditelusuri dalam format
1 : 1
2 : 41
3 : 88
4 : 165
5 : 444
Kedalaman octree: 5
Lama waktu program berjalan: 0.001059 detik
File .obj disimpan: test\line-voxelized.obj
```

**Gambar 5.** Hasil kasus uji 02: berbentuk line dalam tampilan CLI.

### b. Preview dengan *Blender*



**Gambar 6.** Hasil kasus uji 02: berbentuk line dalam tampilan *Blender*.

### 3. *Kasus 03: Testcase Pumpkin*

Pada pengujian ketiga, digunakan model labu (pumpkin-voxelized.obj) dengan batas kedalaman maksimum Octree sebesar 5. Pada tingkat kedalaman ini, program melakukan evaluasi terhadap total 8.816 node dalam struktur Octree. Berdasarkan statistik kami diperoleh, proses pruning menunjukkan kinerja yang cukup signifikan dengan mengeliminasi 4.278 node kosong. Setelah tahap tersebut, tersisa 4.538 node yang dinyatakan valid untuk diproses lebih lanjut. Node-node valid ini kemudian diekstraksi menjadi 4.538 kubus voxel, yang secara keseluruhan mampu merepresentasikan siluet dan bentuk asli model labu.

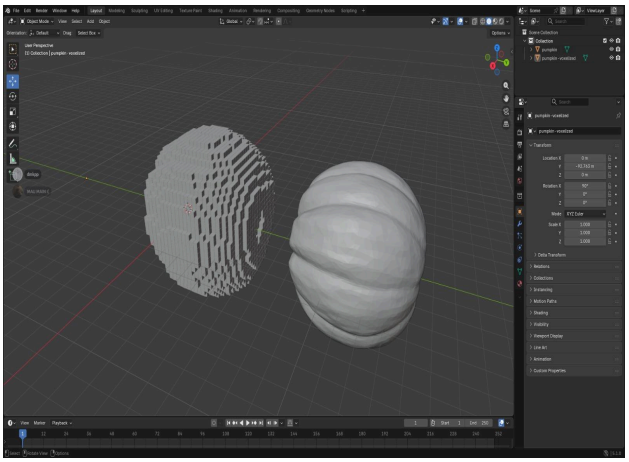
#### a. Tampilan hasil (CLI)

```
PS C:\smthshit\tucil> .\voxelizer test\pumpkin.obj 5
[INFO] Parsing selesai: 5002 vertices, 10000 triangles.
[INFO] Ekspor selesai: 4538 voxels -> test\pumpkin-voxelized.obj

Banyaknya voxel yang terbentuk: 4538
Banyaknya vertex yang terbentuk: 36304
Banyaknya faces yang terbentuk: 54456
Statistik node octree yang terbentuk dalam format
1 : 8
2 : 64
3 : 448
4 : 2216
5 : 8816
Statistik node yang tidak perlu ditelusuri dalam format
1 : 0
2 : 8
3 : 171
4 : 1114
5 : 4278
Kedalaman octree: 5
Lama waktu program berjalan: 0.009048 detik
File .obj disimpan: test\pumpkin-voxelized.obj
```

**Gambar 7.** Hasil kasus uji 03: berbentuk pumpkin dalam tampilan CLI.

#### b. Preview dengan *Blender*

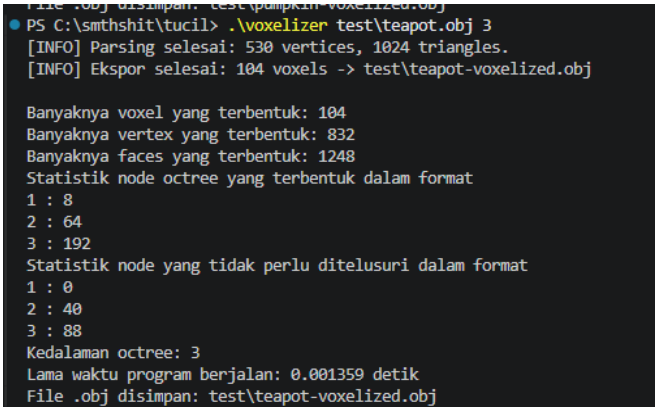


**Gambar 8.** Hasil kasus uji 03: berbentuk pumpkin dalam tampilan *Blender*.

## 4. *Kasus 04: Testcase Teapot*

Pada pengujian keempat, digunakan model teko (teapot-voxelized.obj) dengan batas kedalaman maksimum Octree sebesar 3. Pada tingkat kedalaman ini, program melakukan evaluasi terhadap total 192 node dalam struktur Octree. Berdasarkan statistik yang diperoleh, proses pruning berhasil mengeliminasi 88 node yang tidak memiliki informasi (node kosong), sehingga menunjukkan efisiensi dalam penyaringan data yang tidak relevan. Setelah proses tersebut, tersisa 104 node yang dinyatakan valid untuk diproses lebih lanjut. Node-node valid ini kemudian diekstraksi menjadi 104 kubus voxel, yang secara keseluruhan mampu merepresentasikan bentuk dan siluet asli model teko dengan cukup baik serta tetap mempertahankan karakteristik geometrinya.

### a. Tampilan hasil (CLI)

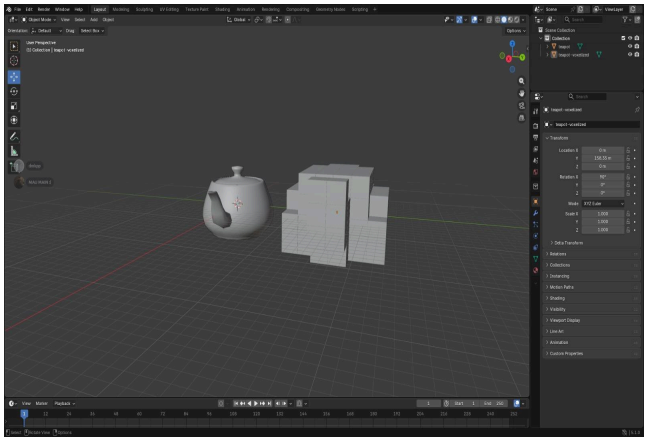


```
PS C:\smthshit\tucil> .\voxelizer test\teapot.obj 3
[INFO] Parsing selesai: 530 vertices, 1024 triangles.
[INFO] Ekspor selesai: 104 voxels -> test\teapot-voxelized.obj

Banyaknya voxel yang terbentuk: 104
Banyaknya vertex yang terbentuk: 832
Banyaknya faces yang terbentuk: 1248
Statistik node octree yang terbentuk dalam format
1 : 8
2 : 64
3 : 192
Statistik node yang tidak perlu ditelusuri dalam format
1 : 0
2 : 40
3 : 88
Kedalaman octree: 3
Lama waktu program berjalan: 0.001359 detik
File .obj disimpan: test\teapot-voxelized.obj
```

**Gambar 9.** Hasil kasus uji 04: berbentuk teapot dalam tampilan CLI.

### b. Preview dengan *Blender*



**Gambar 10.** Hasil kasus uji 04: berbentuk teapot dalam tampilan *Blender*.

## 5. *Kasus 05: Testcase Male (generate oleh kelompok kami)*

Pada pengujian Ke 5 kami, digunakan model Male (male-voxelized.obj) dengan batas kedalaman maksimum Octree sebesar 4. Pada tingkat kedalaman ini, program melakukan evaluasi terhadap total 448 node dalam struktur Octree. Berdasarkan hasil statistik yang diperoleh, terlihat bahwa proses pruning berjalan efektif dengan mengeliminasi 250 node yang tidak mengandung informasi (node kosong). Dengan demikian, tersisa 86 node yang valid untuk diproses lebih lanjut. Node-node valid tersebut kemudian diekstraksi menjadi 198 kubus voxel, yang secara keseluruhan mampu merepresentasikan bentuk dan siluet asli dari model pria dengan cukup baik.

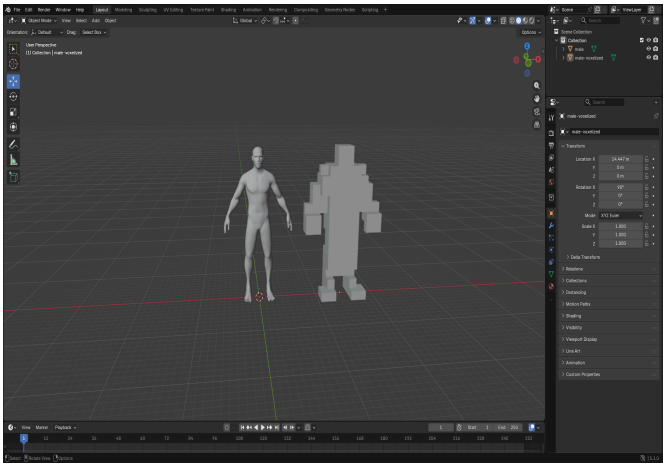
### a. Tampilan hasil (CLI)

```
PS C:\smthshit\tucil> ./voxelizer test/male.obj 4
[INFO] Parsing selesai: 24461 vertices, 24459 triangles.
[INFO] Ekspor selesai: 198 voxels -> test\male-voxelized.obj

Banyaknya voxel yang terbentuk: 198
Banyaknya vertex yang terbentuk: 1584
Banyaknya faces yang terbentuk: 2376
Statistik node octree yang terbentuk dalam format
1 : 8
2 : 64
3 : 192
4 : 448
Statistik node yang tidak perlu ditelusuri dalam format
1 : 0
2 : 40
3 : 136
4 : 250
Kedalaman octree: 4
Lama waktu program berjalan: 0.021324 detik
File .obj disimpan: test\male-voxelized.obj
```

**Gambar 11.** Hasil kasus uji 05: berbentuk male dalam tampilan CLI.

### b. Preview dengan *Blender*



**Gambar 12.** Hasil kasus uji 05: berbentuk male dalam tampilan *Blender*.

# 5. Kesimpulan

Berdasarkan proses perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa program *voxelization* ini berhasil dibangun menggunakan metode *Divide and Conquer* melalui struktur data *Octree*. Pendekatan ini terbukti sangat efektif untuk menangani permasalahan komputasi spasial 3D. *Octree* memampukan program untuk membagi ruang menjadi sub-ruang kecil dan secara cerdas mengabaikan area yang kosong (*pruning*), sehingga memori dan waktu komputasi menjadi jauh lebih efisien.

Selain itu, penerapan fitur *concurrency* (pemrosesan paralel) turut memberikan peningkatan performa yang signifikan, terutama saat memproses model 3D dengan jumlah poligon masif. Secara keseluruhan, program telah memenuhi semua spesifikasi yang ditentukan. Hasil akhirnya berupa representasi *voxel hollow* (hanya membentuk permukaan luar objek) yang terjamin validitasnya; di mana seluruh sisi kubus telah ditriangulasi menjadi wajah segitiga dan terbebas dari penulisan indeks negatif. Dengan demikian, *file* .obj luaran yang dihasilkan dapat dirender dan ditampilkan dengan sempurna menggunakan perangkat lunak grafis 3D seperti Blender.

## 6. Daftar Pustaka

[1]. **Tim Asisten IF2211**, *"Spesifikasi Tugas Kecil 2 IF2211 Strategi Algoritma: Voxelization Objek 3D menggunakan Octree,"* Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika Institut Teknologi Bandung, 2026.

[2]. **M. Rinaldi**, *"Algoritma Divide and Conquer (Bagian 1),"* Bahan Kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika Institut Teknologi Bandung, 2026.

[3]. **M. Rinaldi**, *"Algoritma Divide and Conquer (Bagian 2),"* Bahan Kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika Institut Teknologi Bandung, 2026.

[4]. **M. Rinaldi**, "*Algoritma Divide and Conquer (Bagian 3),*" Bahan Kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika Institut Teknologi Bandung, 2026.

[5]. **Maulidevi N. U.** dan **M. Rinaldi**, "*Algoritma Divide and Conquer (Bagian 3),*" Bahan Kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika Institut Teknologi Bandung, 2026.

# 7. Lampiran

## 7.1. Kriteria Program

| No | Poin | Ya | Tidak |
|---|---|---|---|
| 1. | Program berhasil di kompilasi tanpa kesalahan | ✓ | |
| 2. | Program berhasil di jalankan | ✓ | |
| 3. | Hasil konversi .obj terdiri dari kubus-kubus kecil | ✓ | |
| 4. | Program informasi-informasi atau report | ✓ | |
| 5. | Program dapat menerima masukan .obj, dan mengembalikan output | ✓ | |
| 6. | [Bonus] Program menggunakan concurrency | ✓ | |
| 7. | [Bonus] Program dapat menampilkan visualisasi .obj | | ✓ |

## 7.2. Pernyataan Tidak Melakukan Kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Daniel

Daniel Putra Rywandi S.

Dzakwan

Dzakwan Muhammad Khairan Putra Purnama

****Oleh:** *Daniel Putra Rywandi S. dan Dzakwan Muhammad Khairan P. P.*

*Link Source Code Program (github) kami:* [https://github.com/ItsMeD4N/Tucil2_13524143_13524145](https://github.com/ItsMeD4N/Tucil2_13524143_13524145)